EE6332 Modelling And Optimization In VLSI

End-Sem Project Report

Submitted By: Aman Saini (ee24s007)

Phase I: Continuous Gate Sizing Problem

1. T_{WALL} and Area minimum solutions

The following table illustrates the minimum-delay solution (T_{WALL}), and minimum area solution for a given timing specification obtained using GPkit and mosek.

Circuit	T_{WALL}	Area-minimum solution					
		1.05x T_{WALL}	1.10x T_{WALL}	1.20x T_{WALL}	1.30x T_{WALL}	1.40x T_{WALL}	1.50x T_{WALL}
C17	28.83	199.13	182.72	156.33	136.11	120.20	107.39
C432	153.47	1009.86	782.78	567.12	460.31	397.88	358.31
C880	118.93	1305.59	1150.88	984.85	891.27	831.45	789.37
C1908	177.58	2429.10	2035.23	1697.97	1531.48	1437.45	1377.09
C2670	178.69	3512.78	3140.01	2861.28	2708.75	2605.16	2527.71
C3540	220.71	3279.49	2990.67	2810.35	2734.58	2692.97	2665.80
C5315	203.07	5123.67	4770.46	4440.02	4265.34	4151.61	4069.35
C6288	564.65	3716.74	3277.07	2949.71	2825.94	2769.08	2738.74
C7552	167.87	7372.77	6768.99	6253.22	5995.34	5834.90	5722.54

Table 1: *Summary of continuous gate sizing solutions.*

The column labeled as T_{WALL} shows the minimum possible total delay from input to output. The columns corresponding to area-minimum solution shows the smallest total area (i.e., sum of gate sizes) that satisfies the given timing specification which is defined relative to T_{WALL}.

The following figure shows the plot of T_{WALL} against the number of gates in the circuit.

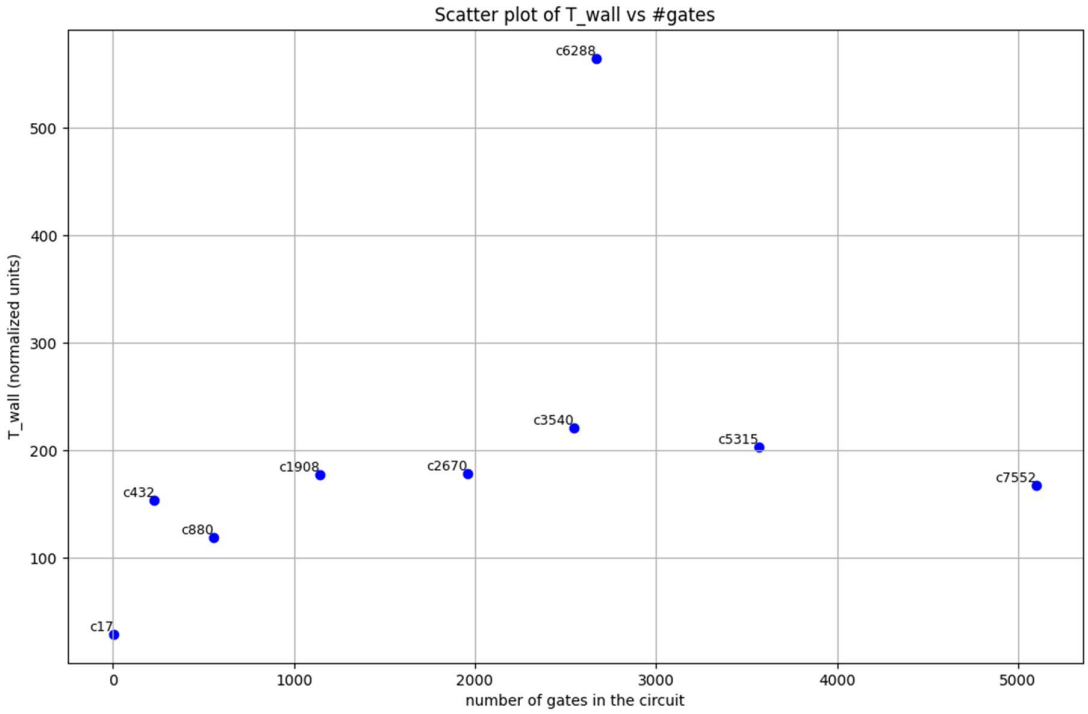


Figure 1: *Plot of T_{WALL} vs number of gates for different benchmark circuits*

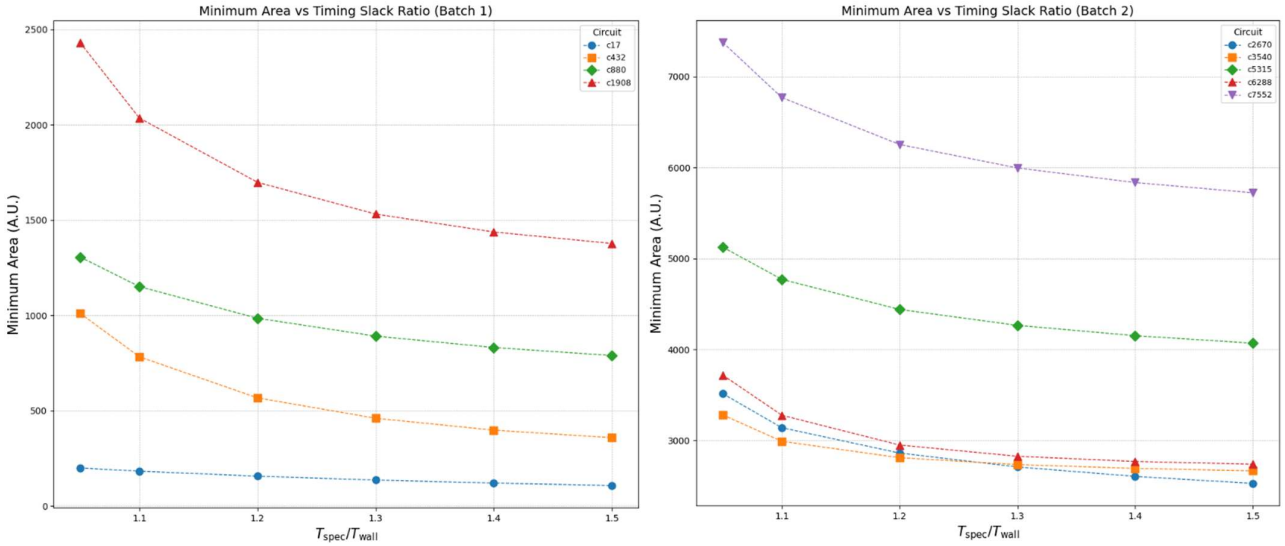


Figure 2: *Plot of A_{MIN} vs T_{SPEC} / T_{WALL} for all ISCAS85 benchmark circuits. The circuits are divided into separate batches based on their total area for better visualization clarity.*

2. Critical and non-critical paths

A critical path can be found out by computing slack for each node in the circuit, then tracing a path from input to output with slack on every node equal to 0. The opposite can be done to find a non-critical path, for which some nodes have non-zero slacks.

--- Solution Details ---
ISCAS85 Benchmark ckt: c7552
T_delay: 218.226; (Area Optimized; T_spec = 218.226)

--- Slack for Critical Path (Topological Order) ---

S.No	Node	Gate	Size	Node Type	Slack
6	N18	_	_	Input	0.0
220	temp50	NOT1_214	1.22		0.0
331	N689	NOT1_226	1.65		0.0
468	N1206	NOT1_1292	1.21		0.0
580	temp222	NOT1_4511	1.0		0.0
735	N2045	NOT1_4512	1.05		0.0
900	N2412	NOT1_4796	1.0		0.0
1094	temp470	NAND2_153	1.0		0.0
1224	N3037	NOT1_154	1.0		0.0
1355	temp570	NOR2_383	1.0		0.0
1520	N3466	NOT1_384	1.0		0.0
1871	temp841	NOT1_1211	1.0		0.0
2224	N5086	NOT1_1212	1.0		0.0
2730	N6684	NAND2_1764	1.0		0.0
2964	N7358	NAND2_2100	1.25		0.0
3346	n_3	NAND4_5100	1.0		0.0
3696	n_4	NOR2_5099	1.0		0.0
3959	n_5	NOR3_5097	1.0		0.0
4117	N9035	NAND2_5096	1.0		0.0
4240	temp1308	NAND2_3232	1.0		0.0
4340	N9618	NOT1_3233	1.0		0.0
4429	temp1333	NOR2_3364	1.0		0.0
4501	N9820	NOT1_3365	1.16		0.0
4567	temp1341	NAND4_3477	1.0		0.0
4635	N10017	NOT1_3478	1.0		0.0
4673	n_65	NOR3_5017	1.06		0.0
4710	N10102	NAND2_5013	10.44	Output	0.0

--- Slack for Non-Critical Path (Topological Order) ---

S.No	Node	Gate	Size	Node Type	Slack
133	N209	_	_	Input	12.485
701	temp197	NAND2_4431	1.0		0.0
840	N1957	NOT1_4432	1.0		0.0
1019	temp297	NOR2_4706	1.0		0.0
1174	N2348	NOT1_4707	1.0		0.0
1305	temp382	NAND2_4907	1.0		0.0
1465	N2619	NOT1_4908	1.0		0.0
1747	N3515	NOR2_411	1.0		11.191
2101	N4795	NOT1_998	1.0		11.191
4214	N9067	NAND2_2907	1.0		1.137
4413	N9642	NAND2_3250	1.0		0.0
4479	N9958	NAND2_3433	1.0		0.645
4620	N10170	NAND2_3602	1.0		0.0
4668	N10315	NAND2_3699	1.0		0.45
4754	N10509	NAND2_3786	1.0		0.0
4797	N10672	NAND2_3915	1.0		1.137
4891	N10789	NAND2_4019	1.0		0.0
4950	N10873	NAND2_4075	1.0		1.147
5063	N10928	NAND2_4119	1.0		0.0
5099	N10989	NAND2_4146	1.0		1.147
5159	N11044	NAND2_4185	1.0		0.0
5180	N11115	NAND2_4219	1.0		1.158
5222	N11168	NAND2_4256	1.0		0.0
5239	N11214	NAND2_4275	1.0		1.148
5262	N11260	NAND2_4317	1.0		0.0
5273	temp1486	NAND2_4330	1.0		0.0
5280	N11278	NOT1_4331	1.0		0.0
5285	temp1491	NOR2_4357	1.0		0.0
5290	N11299	NOT1_4359	1.06		0.0
5295	N11313	NAND2_4368	1.0		0.315
5305	N11328	NAND2_4382	3.71		0.0
5308	N11334	NOT1_4386	27.41	Output	0.0

*The tables above show the critical and non-critical paths obtained in the circuit **c7552** for the area optimized solution (T_{SPEC} = 1.2x T_{WALL}).*

In the critical path, the slack of every node is 0, i.e., the arrival time of the signal is exactly equal to required arrival time to meet the timing specification.

In the non-critical paths, it can be observed that some nodes are associated with a positive slack, and the gate driving that node is unit-sized. It can be attributed to the fact that if it were not, then it would have been possible to downsize that gate, without violating the T_{SPEC} which is not possible as the solution is already min. area for that given T_{SPEC}.

Similar observations can be made for rest of the circuits in the benchmark suite.

```
--- Solution Details ---
ISCAS85 Benchmark ckt: c432
T_delay: 199.513; (Area Optimized; T_spec = 199.513)

--- Slack for Critical Path (Topological Order) ---
+------+------+-----------+-------+-----------+-------+
| S.No | Node |   Gate    | Size  | Node Type | Slack |
+------+------+-----------+-------+-----------+-------+
|  28  | N89  |     _     |   _   |   Input   |  0.0  |
|  51  | N146 |  NOT1_45  | 1.77  |           |  0.0  |
|  76  | N177 | NAND2_59  | 1.89  |           |  0.0  |
| 117  | n_48 | NAND2_173 |  1.0  |           |  0.0  |
| 129  | N247 | NAND2_153 | 1.06  |           |  0.0  |
| 142  | N282 | NAND2_101 | 1.48  |           |  0.0  |
| 200  | n_15 | NOR2_215  |  1.0  |           |  0.0  |
| 211  | n_20 | NOT1_210  |  1.0  |           |  0.0  |
| 220  | N341 | NAND2_192 | 1.05  |           |  0.0  |
| 224  | N355 |  NAND2_7  | 1.49  |           |  0.0  |
| 227  | n_67 | NAND3_144 | 2.32  |           |  0.0  |
| 229  | N357 | NOR3_142  | 3.67  |           |  0.0  |
| 231  | N370 |  NOT1_10  | 11.28 |  Output   |  0.0  |
+------+------+-----------+-------+-----------+-------+

--- Slack for Non-Critical Path (Topological Order) ---
+------+------+-----------+-------+-----------+--------+
| S.No | Node |   Gate    | Size  | Node Type | Slack  |
+------+------+-----------+-------+-----------+--------+
|  2   |  N4  |     _     |   _   |   Input   | 5.578  |
|  55  | N154 | NAND2_49  |  1.0  |           | 1.917  |
| 116  | n_49 | NAND2_172 |  1.0  |           | 1.473  |
| 128  | N224 | NAND2_151 |  1.0  |           | 1.473  |
| 141  | N263 | NAND2_93  |  1.0  |           | 12.003 |
| 157  | N300 | NOT1_112  |  1.0  |           | 12.003 |
| 217  | N348 | NAND2_135 | 1.17  |           |  0.0   |
| 225  | n_68 | NAND4_143 | 2.01  |           |  0.0   |
| 229  | N357 | NOR3_142  | 3.67  |           |  0.0   |
| 231  | N370 |  NOT1_10  | 11.28 |  Output   |  0.0   |
+------+------+-----------+-------+-----------+--------+
```

The table above shows the critical and non-critical paths for the circuit **c432**. As observed, the gates with non-zero slack are all assigned unit sizes to achieve area optimality.

DISCRETIZATION OF GATE SIZES

A breadth-first-search variant of branch & bound was used to discretize the gate sizes, such that it satisfies the given timing constraint. The basic idea was to explore the no more than ***k*** nodes (corresponding to the ***k*** lowest delays from primary inputs to primary output) at each depth level to guide the search efficiently to the optimal solutions.

After the discretization using branch and bound, a local refinement step is performed if the resulting solution does not satisfy the timing specification. For each gate, the circuit delay is evaluated for gate sizes in the vicinity of the current size, typically from (current size – 2) to (current size + 2). The gate is then assigned the size within this range that yields the lowest total delay. This process is repeated iteratively until the timing specification is satisfied

```
T_cont = solver.solution['variables']['T0'] - 1 # delay (continuous gate sizing)

# perform BFS, explore maximum of 'srch_idx' branches on every level

curr_level = [] # stores the solutions at the current level
curr_level.append((T_cont, cont_sol_dict))

for node in tmp_G.nodes:
    node_data = tmp_G.nodes[node]
    if(node_data['type'] != 'input'):
        curr_level = sorted(curr_level, key = lambda x: x[0]) # sorting the solutions in curr level based on the T_delay
        nxt_level = []
        for i in range(min(srch_idx, len(curr_level))):
            tmp_sol_new = curr_level[i][1]
            if(abs(tmp_sol_new[node] - math.floor(tmp_sol_new[node])) < 1e-3):
                # if the gate size is already close to an integer value, just round the gate size and continue
                tmp_sol_new[node] = round(tmp_sol_new[node])
                T_new = get_delay_from_sizes(solver, tmp_sol_new)
                nxt_level.append((T_new, tmp_sol_new))
                continue
            # else explore both floor and ceil
            # floor
            tmp_sol_new[node] = math.floor(tmp_sol_new[node])
            T_new = get_delay_from_sizes(solver, tmp_sol_new)
            nxt_level.append((T_new, tmp_sol_new))

            # ceil
            tmp_sol_new[node] += 1
            T_new = get_delay_from_sizes(solver, tmp_sol_new)
            nxt_level.append((T_new, tmp_sol_new))

        curr_level = nxt_level

# get the best solution from the curr_level_dict
curr_level = sorted(curr_level, key = lambda x: x[0])
best_sol_T = curr_level[0][0]
best_sol_dict = curr_level[0][1]
```

***Code snippet 1**: showing the specifics of branch & bound algorithm used for discretization*

```
for node in tmp_sol:
    if(T_delay <= t_spec):
        break
    best_sz = tmp_sol[node]
    # tmp_sol_new = tmp_sol.copy()
    loops = 0
    while(tmp_sol[node] != 1 and loops != 2):
        tmp_sol[node] -= 1
        T_new = get_delay_from_sizes(solver, tmp_sol)
        if(T_new <= T_delay):
            T_delay = T_new
            best_sz = tmp_sol[node]
            # tmp_sol = tmp_sol_new.copy()
        loops += 1
    tmp_sol[node] += loops
    loops = 0
    while(tmp_sol[node] != solver.max_gate_size and loops != 2):
        tmp_sol[node] += 1
        T_new = get_delay_from_sizes(solver, tmp_sol)
        if(T_new <= T_delay):
            T_delay = T_new
            best_sz = tmp_sol[node]
            # tmp_sol = tmp_sol_new.copy()
        loops += 1
    tmp_sol[node] = best_sz
```

***Code snippet 2:** shows a post-processing function that locally refines the gate sizes obtained from branch & bound.*

More specifically, for each gate, it evaluates the delay for the gate size in range [current size – 2, current size + 2], and updates the gate size to the one yielding the minimum delay.

This refinement is performed twice: once in topological order and once in reverse topological order of the gates in the circuit graph.

The following table presents the delays obtained using various heuristic discretization schemes alongside the delay obtained using branch & bound. Additionally, it includes the delay corresponding to the continuous gate sizing solution ($T_{SPEC} = 1.3x\ T_{WALL}$), which serves as the timing specification (T-spec) that a discrete solution must satisfy

Circuit	Continuous solution	Discrete Solutions			
		Floor	Ceil	Round	Branch & bound
C17	37.48	38.22	36.95	38.22	36.95
	136.11	134	140	135	140
C432	199.51	233.50	207.62	212.25	198.27
	460.31	428	657	459	494
C880	154.61	167.86	162.23	163.52	154.40
	891.27	862	1417	888	932
C1908	230.86	246.89	235.64	239.41	230.76
	1531.48	1496	2640	1511	2088
C2670	232.30	282.17	242.71	256.00	230.57
	2708.75	2607	4565	2676	2836
C3540	286.92	351.33	297.79	308.67	283.33
	2734.58	2712	5260	2731	2775
C5315	263.99	345.33	277.58	285.31	261.00
	4265.34	4164	7733	4269	9958
C6288	734.04	1225.67	764.72	1065.67	728.83
	2825.94	2783	5455	2822	2940
C7552	218.22	266.33	229.51	231.52	217.33
	5995.34	5873	10976	5989	6147

***Table 2**: Comparison of different discretization schemes. The yellow row shows the **normalized delay**, while the white rows indicate the **total area** (i.e., the sum of gate sizes).*

From the above table it can be seen, that the heuristic discretization schemes fail to meet the timing specification defined by the continuous gate sizing solution. On the other hand, branch and bound algorithm satisfies the timing specification for each circuit.

AREA OPTIMIZATION

This means, that we can take the solution obtained using the described algorithm, and specifically pass it through a function that optimizes it for area. This can be done by reducing the size of the gates with positive slack.

The function iteratively traverses through each gate of the circuit, and attempts to reduce its size provided it does not lead to a violation of the timing constraint. Multiple such passes are typically performed to progressively lower the area.

```python
def area_clipper(solver, sizes_sol, permit):
    """
    Reduces gate sizes to minimize area while allowing a timing penalty
    of up to 'permit' units.

    Parameters:
        solver     : the solver object containing graph and solution info
        sizes_sol  : dict mapping each gate to its current size
        permit     : maximum allowed increase in delay (relative to initial T_curr)

    Returns:
        T_curr     : new total delay after clipping
        sizes_sol  : updated size solution
    """
    tmp_G = solver.G
    Tspec = solver.solution['variables']['T0'] - 1  # timing constraint
    T_curr = get_delay_from_sizes(solver, sizes_sol)

    for node in tmp_G.nodes:
        if tmp_G.nodes[node]['type'] == 'input':
            continue
        if sizes_sol[node] > 1:
            sizes_sol[node] -= 1
            T_tmp = get_delay_from_sizes(solver, sizes_sol)
            if T_tmp <= Tspec and (T_tmp - T_curr) < permit:
                T_curr = T_tmp
            else:
                sizes_sol[node] += 1  # revert

    return T_curr, sizes_sol
```

Code Snippet 3: *Area clipper function, reduces the size of the gates, if the resulting delay increase is within a permitted threshold.*

```python
def min_area_ctrl(solver, sizes_sol):

    tmp_G = solver.G
    Tspec = solver.solution['variables']['T0'] - 1
    T_curr = get_delay_from_sizes(solver, sizes_sol)

    area_optimal = sizes_sol.copy()

    # Compute initial permitted slack margin
    permit = round((Tspec - T_curr) / 100, 4)
    if Tspec < T_curr:
        permit = 0  # T_delay already more that Tspec. (only reduce the sizes with no penalty in delay)

    # Case 1: only reduce sizes if delay does not increase at all
    if permit == 0:
        while True:
            T_tmp, area_optimal, clipped = area_clipper(solver, area_optimal, permit=0)
            if T_tmp == T_curr:
                break
            T_curr = T_tmp

    # Case 2: gradually increase permit to find the sweet spot
    while permit < (Tspec - T_curr):
        T_tmp, area_optimal= area_clipper(solver, area_optimal, permit)
        if T_tmp == T_curr:
            permit *= 2
        else:
            T_curr = T_tmp
    while (permit != 0):
        T_tmp, area_optimal= area_clipper(solver, area_optimal, permit)
        if T_tmp == T_curr:
            permit = round(permit/2, 4)
        else:
            T_curr = T_tmp

    # Compute final area
    sum_sizes = sum(area_optimal[node]
                    for node in tmp_G.nodes
                    if tmp_G.nodes[node]['type'] != 'input')

    return area_optimal, sum_sizes, T_curr
```

Code Snippet 4: *Area optimizer function. Calls the area clipper function multiple times, dynamically controlling the permitted delay penalty, to reduce the sizes of as much gates as possible, while staying within the overall delay constraint*

The following table illustrates the result, comparing the branch and bound solution to the area-optimized solution. It also reports the percentage area increase relative to the continuous solution, and the total runtime to obtain the area-optimized discrete solution starting from a continuous one.

Circuit	# gates	Continuous solution	Discrete Solution		% area increase	Runtime (seconds)
			Branch & bound	Area-optimized		
C17	**6**	37.48	36.95	37.43	1.38	0.004
		136.11	140	138		
C432	**229**	199.51	198.27	199.48	4.71	1.655
		460.31	494	482		
C880	**555**	154.61	154.40	154.43	2.55	8.186
		891.27	932	914		
C1908	**1144**	230.86	230.76	230.82	2.64	54.49
		1531.48	2088	1572		
C2670	**1958**	232.30	230.57	231.88	0.01	176.81
		2708.75	2836	2709		
C3540	**2548**	286.92	283.33	286.79	0.67	223.32
		2734.58	2775	2753		
C5315	**3569**	263.99	261.00	263.78	1.82	830.22
		4265.34	9958	4343		
C6288	**2672**	734.04	728.83	733.83	1.13	242.94
		2825.94	2940	2858		
C7552	**5103**	218.22	217.33	218.16	1.19	1034.65
		5995.34	6147	6067		

Table 3: *Comparison of the branch-and-bound solution with the area-optimized solution. The algorithm's runtime remains within practical limits, and the area increase is contained within 5% relative to the continuous solution.*

The yellow row shows the normalized delay, while the white rows indicate the total area

Comparison With CVXPY Mixed-Integer Geometric Program

A mixed integer problem requires the gate-size variable to be declared as an integer. To obtain the solution, when one of the variables does not take a continuous positive real value, the problem should be setup in such a way that it obeys the rules of disciplined convex programming (DCP).

To address this, an auxiliary variable is introduced to store the logarithm of gate sizes. This variable is then used to express the constraints associating the arrival time at the output of a gate to the input arrival times in the following form:

$$AT_{out} = AT_{in} + pd + e^{(\log[LE_i * \sum_{i \in FanOut(j)} e^{s_i}] - s_j)}$$

Or, when the gate drives a primary output:

$$AT_{out} = AT_{in} + pd + e^{(\log[LE_i * \sum_{i \in FanOut(j)} e^{s_i} + C_{out}] - s_j)}$$

Here, pd denotes the parasitic delay of gate j. The variable s denotes the log size variable such that:

$$e^{s_i} = x_i$$

where x_i is the gate size of i[th] gate.

Given below is the code snippet to add the above expressions as constraints of the optimization problem:

```
for fi_node in fan_in_nodes:
    if attr.get('is_output', False):
        log_sum_expr = cp.log_sum_exp(
            cp.hstack([self.log_size[fo_idx_list[i]] + np.log(le) for i, le in enumerate(le_list)] + [np.log(self.out_cap)])
        )
    else:
        log_sum_expr = cp.log_sum_exp(
            cp.hstack([self.log_size[fo_idx_list[i]] + np.log(le) for i, le in enumerate(le_list)])
        )

    pred_idx = G.nodes[fi_node]['n_idx']
    curr_idx = attr['n_idx']
    gate_idx = self.gate_info_map[curr_gate][1]

    self.constraints.append(self.a_t[curr_idx] >= self.a_t[pred_idx] + pd + cp.exp(log_sum_expr - self.log_size[gate_idx]))
```

To ensure, that the $e^{\log_size} = g_size$, a penalty term is introduced in objective function along with the sum of gate sizes.

$$objective = \sum gate_size + w \cdot \sum \left(e^{\log_size} - gate_size\right)^2$$

Here, w is a factor, which can be adjusted to get $e^{\log_size}$ closer to $gate_size$, which is declared as an integer. In the final solution, $e^{\log_size}$ can be rounded off to an integer to obtain a discrete solution.

The following image shows the result of discrete sizing for circuit “c17” obtained by solving the convex optimization problem.

```
Twall = 28.833

Sizes of gates = ['9.268', '28.232', '37.500', '18.750', '64.000', '64.000']
Sum of sizes: 221.75
Results saved to: /content/drive/MyDrive/EE6332/sizing_results/c17/c17_minT_.txt

================================================================================
--- DISCRETE SOLUTION ---
Total elapsed time (including setup): 0.5569 seconds

Sum of sizes: 137
Verification (discrete_sizes_delay): 37.46031746031746
Tspec =  37.482
g_size:  [ 6. 14. 18.  9. 45. 45.]
log_size:  [ 5.89469505 13.99679312 17.99349269  8.99595332 44.97544951 44.98653178]
```

It can be seen that $e^{\log_size} \approx gate_size$, and the optimization yields an area of 137 for the circuit “c17”, comparable to 138 obtained from custom algorithm.

However, this formulation of an optimization problem takes significantly longer durations to solve for other circuits in the benchmark suite, resulting in no solution being obtained in a reasonable time.

STANDARD CELL LIBRARY

The discrete solution assigns arbitrary integer sizes to the gates in the specified range (e.g., [1, 64]). However, not all the sizes in this range can realistically be included in the standard cell library for every gate type. In which case, we are required to identify, which sizes should be made a part of standard cell library.

Description:

A straightforward approach to this problem is to collect the gate sizes assigned to a particular gate type (INV, NAND_2, NOR_2, etc.) during the discretization of the continuous solutions. This data can be aggregated over:

- Multiple benchmark circuits
- Various values of T_{SPEC} / T_{WALL} (i.e., different levels of timing relaxations)

We can then define a fractional occurrence threshold, and any size that occurs more frequent than this threshold can be included in the standard cell library.

Since these sizes are obtained from the solutions optimized for area, most of them are expected to lie in the lower end of the range. This would naturally lead to a compact and area efficient library, focused on smaller gate sizes.

Nevertheless, sizes on the higher end of the range are equally important to be included as they can be critical to meet the timing specification. To ensure coverage across entire range [1, 64], it can be partitioned into sub-ranges of 4 consecutive gate sizes each. For each range, that does not already contain a size in the library, most frequently occurring size from that subrange can be included. If no size appears from a subrange, then no size is included.

Following above algorithm, the following table depicts the standard cell library.

Gate type	Sizes**
NOT	[1, 2, 3, 4, 5, 6, 7, 8, 9, 13, 20, 22, 27, 29, **32**, **64**]
NAND_2	[1, 2, 3, 4, 5, 8, 11, 12, 13, 17, **32**, 46, **64**]
NAND_3	[1, 2, 3, 4, **8, 16, 32, 64**]
NAND_4	[1, 2, 3, 4, 6, 8, 9, 11, **16**, 23, 25, **32, 64**]
NOR_2	[1, 2, 3, 4, 5, 6, 9, 13, 18, 27, **32, 64**]
NOR_3	[1, 2, 4, 8, **16, 32, 64**]
NOR_4	[1, 2, **4, 8, 16, 32, 64**]

Note: *standard cell library based on 1.3x timing relaxation factor.*

****Sizes in bold** are appended to the library obtained using the above algorithm. Although no gate size from their corresponding subranges appears in the solutions, these sizes are essential to meet tighter timing specifications.

The initial set of sizes was extracted from solutions optimized at a T_{SPEC}/T_{WALL} ratio of 1.3. To ensure timing flexibility and coverage across the entire range, the following canonical sizes were added, if no nearby size was already present in the extracted set: **1, 2, 4, 8, 16, 32, 64**.

After relaxing the convergence constraints, the following solutions were obtained from CVXPY. It can be seen in the table, that the timing specification for 3 circuits are not met. Some post processing can be done for them, to get the result.

For 2 circuits, the solution obtained using CVXPY yield a better area than the custom algorithm: C17, and C1908.

circuit		CVXPY Mixed Integer GP		Post Processing	
	T_spec	T_delay (discrete sol.)	Sum of sizes	T_delay (discrete sol.)	Sum of sizes
C17	37.48	37.46	137	_	_
C432	199.51	200.48	479	199.36	482
C880	154.6	155.25	913	154.54	915
C1908	230.86	232.25	1561	230.83	1568
C2670	232.30	230.56	2747	_	_
C3540	286.92	286.31	2763	_	_
C5315	263.99	264.00	4400	_	_
C6288	734.04	728.83	2940	_	_
C7552	218.22	217.33	6170	_	_